HIVE — 3-minute walkthrough script

Everything below was verified live on real EnergyPlus 24.1.0 + a real local Ollama (`qwen2.5:7b-instruct-q4_K_M`). Narration is in **plain text**; commands are in code blocks; "[on screen]" notes what to show. Total ≈ 3:00.

> **The honesty rule for this video:** every claim is something you can reproduce on camera. The one thing we do **not** claim is a headline "% energy saved" number — see the *Results & honest limitation* beat (2:20). Leading with the self-heal loop, not a savings figure, is deliberate: it is the part that is unambiguously real on this hardware.

0:00 – 0:25 · Hook + architecture

Say:

> "This is HIVE — a closed-loop building-energy control agent. The idea in one line: *an LLM that plans like an energy manager, wrapped in a deterministic guardian that acts like a controls engineer.* The language model reasons about weather, tariffs and carbon; a hand-written safety layer is the only thing that ever touches an actuator. Here's the shape."

[on screen] `reports/architecture.pdf` page 1 — the mermaid diagram. Trace the loop with the cursor: EnergyPlus → guardian → actuator, and the planner off to the side depositing plans.

0:25 – 1:05 · The closed loop in action — EnergyPlus → LLM → model parameter

> **This is the beat that answers the deliverable directly:** live data from EnergyPlus to the LLM, and the LLM's control action updating a model parameter automatically.

Say:

> "Here's one full cycle of the loop, live. A real EnergyPlus simulation runs, and every timestep it hands the agent a sensor snapshot — temperature, occupancy, comfort."

```
python -m experiments.loop_demo --timeout 200
```

[on screen] point at the **§1** lines as they scroll:

```
EnergyPlus -> agent  |  07-21 06:13  Core_ZN: 24.1 C, 8 occ, PMV -0.33  |  outdoor
```

> "That's live data crossing from EnergyPlus into the agent."

[on screen] §2 — the digest, then the model call:

> "One of those real states is compressed into this digest and sent to the local LLM. It comes back with a plan — and look at the reasoning: the forecast shows a high-tariff morning peak coming, so it chose to **pre-cool now, while power is cheap.** That's the energy-manager judgement we wanted from the model."

[on screen] §3 — guardian verdict + the written parameter:

```
 guardian verdict : accepted
CLGSETP_SCH   24.0 C  ->  24.5 C   [written · drives 5 zone(s)]
```

> "The deterministic guardian approves it, and the setpoint is written straight into the model's schedule — the parameter EnergyPlus uses next timestep. Sensor in, plan out, model updated — automatically, no human in the loop."
>
> **Note for the presenter:** the LLM call takes 25–120s on CPU-only inference (it varies with machine load). Warm the model and close other apps first (see checklist); cut the wait in editing. If you'd rather not wait on camera at all, the self-heal beat below is faster and makes the same point.

1:05 – 1:35 · Self-heal, live — the loop repairing itself

Say:

> "The same loop also repairs the model when something breaks. I'll deliberately break it and watch the agent notice, diagnose, and fix it."

[on screen] run it:

```
python -m experiments.self_heal_demo --timeout 150
```

Say, over the run (≈60–90s — cut the dead air in editing):

> "Step one, it plants a fault — it removes a cooling schedule the thermostat depends on. EnergyPlus now refuses to run: fatal error. Step two, the *real* event detector — the same one a live run uses — sees the Severe error and fires. Step three, that error log goes to the local LLM as a repair prompt, and it emits a structured patch. Step four, the patch is applied through a validate-before-accept primitive and the model re-runs — clean. If it hadn't cleared the error, it would roll back to the last known-good version automatically."

[on screen] point at the final lines:

```
 event    : fired=True reason=severe_error
patch    : repair-xxxxxxxx
outcome  : healed=True rolled_back=False
[PASS] loop resumed (healed)
```

Say:

> "That patch id — `repair-…` — is a real file in a versioned, append-only model history."

1:10 – 1:25 · Self-heal, replay — reliability

Say:

> "For a demo I don't want to depend on the model answering in time, so there's an honest replay mode: it reuses the exact patch from a previous real run — no LLM call — so it's deterministic."

```
python -m experiments.self_heal_demo --replay
```

[on screen] `patch : REPLAY`, `[PASS] loop resumed (healed)`.

1:25 – 1:55 · The guardian — why the LLM is safe to use

Say:

> "The reason I'm willing to put a stochastic model near a building is this deterministic guardian. Let me feed it a hostile plan on purpose — an 18-degree setpoint in an occupied zone, a 5-degree jump, an actuator it doesn't recognise."

```
python -m experiments.smoke_roundtrip --abusive
```

[on screen] the verdict stream — point at the reasons:

```
clip: Core_ZN 18->20 envelope_min | rate: ...->rate_step | strip: ... whitelist
[PASS] clip + rate + strip all fired
```

Say:

> "Clamped to the comfort envelope, rate-limited, and the unknown actuator stripped — and this isn't just tested by example. There's a property-based proof over thousands of adversarial plans of the invariant *'no reachable plan can exit the comfort envelope.*' The safety layer is the claim we test hardest."

1:55 – 2:20 · The ops dashboard

Say:

> "Everything the loop does is observable. This dashboard reads the telemetry database read-only — safe to watch while a run is still writing."

[on screen] `http://localhost:8501` (already running). Scroll slowly: - **Headline** strip — cost / carbon / kWh delta / comfort-violation %. - **Cumulative-kWh race** — three arms, tariff/carbon peak hours shaded. - **Comfort strip** — per-zone PMV inside the ±0.5 envelope. - **Decision journal** — sim-time → trigger → ECMs → guardian verdict, cache-hit vs LLM-call. - **LLMOps** — calls, tokens, p50/p95 latency, verdict counts.

Say:

> "Trigger, the measures it chose, the guardian's verdict, whether it hit the model or the plan cache — every cycle, auditable after the fact."

2:20 – 2:45 · Results & the honest limitation

Say:

> "On the numbers, I'm going to be precise, because it matters. The A/B framework runs three arms under identical conditions and reports real EnergyPlus output — no hand-entered numbers. On this laptop, the honest finding is that a single constrained-decode plan takes tens of seconds on CPU-only inference, while EnergyPlus simulates the whole week in about two seconds — so in the *live* arm the planner can't land a plan before the simulation is over. That's not a savings claim; it's a hardware ceiling. The fix is a GPU-backed model host or the receding-horizon mode, both of which the codebase already supports. I'd rather show you a measurement I trust than a number I can't defend."

[on screen] `reports/results.md` — show the real per-arm table; point at the comfort column (zero occupied violations) as the thing that *is* solid.

2:45 – 3:00 · Close

Say:

> "So: a real closed loop, a safety layer proven against adversarial input, a self-repairing model, full observability — and an honest account of what the hardware does and doesn't let us claim yet. The whole thing is one Python process, a local model, and SQLite. Repo's here, every design decision is written down in CLAUDE.md, and every claim in this video is a command you can re-run."

[on screen] the GitHub repo page + `reports/architecture.pdf`.

Pre-recording checklist (do all of this once, ~10 min before you hit record)

```
 # 1. Env (once per shell)
$env:ENERGYPLUS_DIR = "C:\EnergyPlusV24-1-0"     # PowerShell
$env:OLLAMA_HOST    = "http://localhost:11434"

# 2. Warm the model so the on-camera self-heal call is fast, not a cold 90s load:
ollama run qwen2.5:7b-instruct-q4_K_M "ready"    # then Ctrl-D

# 3. Dashboard up (leave it running in its own window):
python -m dashboard.export_screens               # optional: refresh deck screensho
streamlit run dashboard/app.py

# 4. Do ONE dry run of the self-heal demo so its EnergyPlus files are warm:
python -m experiments.self_heal_demo --replay
```

- Collapse the Streamlit sidebar before recording (top-left arrow) for a cleaner frame.
- Keep `--timeout 150` on the live self-heal call — the default 30s can time out mid-take on CPU-only inference and force a retake.
- Have `reports/architecture.pdf` open to page 1 in a second window for the cold open and close.

What NOT to do on camera

- **Do not quote a "% energy saved" figure.** The live agent arm doesn't actuate on this hardware, and the receding-horizon run's apparent savings is a day-chunking simulation artifact (each chunk cold-starts), not the agent's doing — so it isn't a real control result. The comfort/safety/self-heal story is the one that holds up.
- Don't run `experiments.endurance` live (it's a month/year — show the dashboard endurance card instead).